

Lightweight and Decoupled Distributed Authentication for Multihop IoT Networks

Sreeparna Das, Manas Khatua
Computer Science and Engineering
Indian Institute of Technology Guwahati
{d.sreeparna, manaskhatua}@iitg.ac.in

Abstract—The Internet of Things (IoT) demands lightweight and secure authentication schemes for resource-constrained multihop networks, where IoT nodes communicate with the gateway through multiple intermediary nodes. Present distributed schemes enable secure lightweight authentication for such networks, but overwhelm an IoT node with multiple authentication requests from its newly joined child nodes, leaving other nodes underutilized. This paper presents a decoupled distributed authentication scheme, where IoT nodes capable of authentication are selected by the gateway as an authentication server to authenticate newly joined nodes. The authentication server may be a parent or a non-parent node and thus separates the process of authentication and node joining, mitigating the skewed authentication load by newly joined child nodes to their parent node. After a successful authentication, the authentication server informs the gateway about the authenticated node and mediates the session key generation between the gateway and the node. It utilizes Physically Unclonable Function (PUF) responses to eliminate the need for long-term storage of the secret key (for the secure authentication execution) and prevents adversarial prediction by introducing session-based randomness. Lightweight hash-based accumulators and XOR operations are also introduced, producing reduced computational overhead on resource-constrained nodes. Security validation of the scheme is performed, both formally (using ProVerif) and informally. In addition, performance analysis is conducted through theoretical and simulation-based frameworks, focusing on energy consumption, CPU time, and communication overhead. Analysis demonstrates that the proposed scheme is secure while maintaining a lightweight structure over existing methods.

Index Terms—Lightweight, Authentication, multihop, IoT, and Physically Unclonable Function (PUF).

I. INTRODUCTION

THE Internet of Things (IoT) connects millions of smart devices, collecting and transmitting data to centralized platforms [1], [2] like the gateway through multihop for decision making, for example, industrial IoT [3]. However, IoT nodes are often deployed in outdoor adverse conditions, making authentication inevitable before communication [4]. As these IoT nodes are battery-powered and resource-constrained, the authentication scheme should have a lightweight yet secure design, ensuring efficiency [5], [6].

In the literature, centralized authentication schemes rely on a centralized entity (like the gateway) for the authentication [7], [8], [9], but introduce communication overhead on intermediate nodes while relaying authentication messages from the newly joined node to the authentication server in a multihop environment. This paved the way for distributed schemes that enable

multiple entities (including gateways, edge servers, resource-constrained IoT nodes) in a multihop IoT network to perform authentication. In the distributed scheme [10], every node authenticates its child node. But, in an IoT network, not every node has the capability to authenticate. In addition, child nodes, choosing and joining a particular parent node, may overwhelm that node with multiple authentication requests while other nodes remain underutilized. In [11], although multiple edge servers are employed, the lack of multihop causes child nodes to overload certain servers while others remain underused. These challenges can be mitigated by a decoupled distributed design considering a multihop setting, where an IoT node (capable of authentication and may be a parent or a non-parent node to the newly joined node), selected by the gateway, play the role of an authentication server for that newly joined node. Thus, separates (decouples) the process of authentication and node joining, alleviating the skewed authentication load on parent nodes by their newly joined child nodes.

Designing a decoupled authentication framework for a multihop IoT network poses fundamental challenges. Such a design complicates the synchronization between the gateway, authentication servers, and newly joined nodes. The absence of a central authority forces the resource-constrained node to store secret keys for a long time. Further, multihop communications amplify the risk of attacks, as every additional hop exposes communication to interception, replay, and prediction attempts [12]. In addition, due to the computational and memory limitations of IoT nodes, they lack the capacity to execute computationally intensive cryptographic operations, like encryption/decryption [13], [14], [15]. This challenge becomes even more critical as nodes perform multiple authentications while rejoining the network or establishing a new session, leading to high energy drainage and communication delays [16].

In response to these limitations, this paper proposes a lightweight decoupled authentication scheme that allows gateway-selected IoT nodes (capable of authentication) to authenticate while separating (decoupling) the process of authentication and node joining. Thus, prevents newly joined nodes from overwhelming their parent node with skewed authentication burden. At the end of the authentication, the authentication server securely informs the gateway about the authenticated node, facilitating the generation of a session key between the gateway and the node. The scheme has also incor-

porated a Physically Unclonable Function (PUF), eliminating the long-term storage of the secret key by generating device-specific responses whenever required. A session-based random is additionally introduced to prevent authentication communications from being predicted through interception across multiple hops. Instead of relying on numerous computationally intensive encryption operations, a lightweight hash-based accumulator and XOR operations are included along with two encryption/decryption operations, leading to an efficient authentication scheme that incurs less computational overhead on IoT nodes.

This paper made the following key contributions.

- 1) A lightweight decoupled distributed authentication scheme is introduced for a multihop IoT network, where IoT nodes capable of authentication are selected by the gateway to authenticate newly joined nodes, without overwhelming a particular node.
- 2) The security of the scheme is validated through both informal and formal (using ProVerif) analysis.
- 3) The efficiency of the scheme is evaluated using both theoretical and simulation-based (Cooja-based performance analysis) frameworks, emphasizing communication overhead, energy consumption, and CPU utilization in a multihop constrained setting.

Section II, III, and IV elaborate on related literature, prerequisites, and the proposed scheme, respectively. Furthermore, Section V contains the detailed security analysis. Finally, Section VI presents the performance analysis, with Section VII stating the conclusion.

II. LITERATURE SURVEY

Authentication in IoT multihop networks faces challenges due to its constrained framework and security vulnerabilities. These challenges are alleviated by the inception of distributed schemes, where more than one entity is responsible for authentication, including resource-constrained IoT nodes. In the scheme by Rosa *et al.* [10], parent nodes are authorized by the gateway as an authentication server, authenticating its newly joined child nodes. However, a particular IoT node is overwhelmed with multiple authentication requests by its child nodes. Alongside, Zheng *et al.* [15] has designed a peer-to-peer IoT authentication, where two IoT nodes are authenticating each other without a dependency on a centralized entity. Alruwaili *et al.* [11], on the other hand, utilized multiple edge servers for authenticating IoT nodes, incorporating PUF to design an efficient and attack-resistant scheme. Similarly, PUF-based schemes by Li *et al.* [14] and Yang *et al.* [13] performed authentication and session key exchange without the need for long-term storage of the secret key. Recent works also include a lightweight authenticated-key exchange protocol [17], achieving a lightweight authentication for CoAP/OSCORE, incurring lesser resource overhead. They also include identity-based/certificateless authentication methodology [18], which has eliminated the need for certificate storage by replacing it with Elliptic Curve Cryptography (ECC) and hash-based

primitives. Blockchain is also incorporated in authentication schemes [19], [20], [21] to obtain a decentralized design.

Thus, in this paper, a lightweight decoupled distributed authentication scheme is introduced, which permits gateway-authorized IoT nodes (capable of authentication) to authenticate newly joined child nodes, preventing child nodes from imposing biased authentication load on their parent node. After a successful authentication, the authentication server securely notifies the gateway about the newly authenticated node. The authentication server also facilitates the session key generation between the gateway and the node, managing the synchronization between them. The entire authentication is achieved by incorporating lightweight operations—a hash-based accumulator and XOR. PUF is also considered to eliminate the need for the storage of the secret key. Additionally, a session-based random is used to incorporate unpredictability in authentication communications, updating the authentication secret (incorporating secrecy in authentication communication). Table I highlights key features in [11], [13], [14], [15], [10], and the proposed scheme. These five representative schemes include parent node-based [10], node-to-node PUF-based [22], [14], edge-enabled PUF-based [11], and ECC-based [13] distributed schemes. Thus, incorporates dynamic ways to perform authentication in a distributed manner, establishing a balanced benchmark for evaluating the proposed scheme. From the table, it is apparent that the existing schemes only fulfill a subset of key requirements. Some schemes, such as [13] and [14], integrate decoupled and/ PUF-based authentication, while others, like [11], solely depend on PUF for authentication. Yet they incur high computational overhead due to their reliance on heavy cryptographic operations. On the other hand, [15] obtains a moderate improvement due to the incorporation of PUF, but still suffers from a medium computational burden. In addition, schemes like [10] do not support decoupled architecture, PUF, accumulators, and authentication secret updates, and at the same time exhibit significant computational and communication costs. In contrast, the proposed scheme jointly incorporates decoupling and a hash-based accumulator. It also introduces PUF responses and a session-based random providing secrecy and unpredictability to authentication communications, yet ensures minimal computational and communication load, making it efficient for multihop IoT networks.

TABLE I: Features Comparison of Authentication Schemes.

Scheme	Decoupled	PUF-based	Acc. based	ASU	Comp. OH	Comm. OH
[11]	×	✓	×	✓	High	Low
[13]	✓	×	×	×	High	Low
[14]	✓	✓	×	✓	High	Low
[15]	✓	✓	×	✓	Medium	Low
[10]	×	×	×	×	High	High
Proposed Scheme	✓	✓	✓	✓	Low	Low

Legend: ✓ = Achieved, × = Not Achieved.

Acc. based: One-way Hash-based Accumulator.

ASU: Updation of authentication secret.

Comp. OH: Computational Overhead — Low (Hash/XOR/2 Symmetric Encryption/Decryption), Medium (>2 Symmetric Encryption/Decryption), High (Heavy ECC/Pairings).

Comm. OH: Message exchanges — Low (1–3), Medium (4–5), High (>5).

III. PREREQUISITES

A. Physically Unclonable Function (PUF)

PUF operates as a one-way function performing a mapping of a challenge C to a unique response R from a pool of challenge space $\mathbb{C}$. This uniqueness stems from the inherent hardware variation in IoT nodes, which is infeasible to clone or replicate. To recognize a function as a valid PUF, the following criteria must hold:

- *Evaluatable*: A node embedded with a legitimate PUF can efficiently calculate the response: $R = PUF(C), \forall C \in \mathbb{C}$
- *Unpredictability*: An attacker node without access to a legitimate PUF cannot guess $R \in \mathbb{R}$ from a challenge $C \in \mathbb{C}$ better than a random: $Pr[R = PUF(C)] \approx 1/|\mathbb{R}|$.
- *Uniqueness*: Two nodes with different PUF instances (PUF_1 and PUF_2), never produces same response for a given challenge C : $Pr[PUF_1(C) = PUF_2(C)] \approx 0$
- *Reliability*: A node embedded with a PUF produces the same response R for same challenge C , even at different times: $(R_1 = PUF(C) \& R_2 = PUF(C)) \implies (R_1 = R_2)$
- *One-Wayness*: The inversion of a given response R to obtain the challenge C is infeasible: $\nexists (PUF^{-1}(R) = C)$.

B. Hash-based Accumulator

A hash-based accumulator is a lightweight cryptographic construct [23], where multiple elements are hashed and combined in a single compact value. It allows individual elements to prove their membership within the accumulated value without showing other values. In a one-way accumulator, the order of accumulating input elements does not alter the final combined value. This is because of its quasi-commutative property, which is defined by a function $f : A \times B \Rightarrow A$, such that, for all $a \in A$, and $b, c \in B$,

$$f(a, f(b, c)) = f(f(a, b), c) \quad (1)$$

The above Equation 1 implies that the accumulation order does not impact the final value, where a is considered to be the seed. A formal definition of the hash-based one-way accumulator is defined as follows: Let $Y = \{y_1, y_2, \dots, y_m\}$ be a set of elements, where each element y_i is hashed (Equation 2), where for all $i \in [1, m]$,

$$Y_i = H(y_i) \quad (2)$$

Then, these hashed values are accumulated into one compact value as in Equation 3, where s is considered as the seed and $\&$ is the bitwise AND operation:

$$H(s, Y) = s \&_{i=1}^m Y_i \quad (3)$$

During the membership verification, the hash of the individual element is recomputed and checked as in Equation 4:

$$H(s, Y) \& H(y_i) = H(s, Y) \quad (4)$$

If Equation 4 holds, then y_i is the member of the accumulated value $H(s, Y)$.

IV. PROPOSED AUTHENTICATION SCHEME

The proposed decoupled distributed authentication scheme for multihop IoT networks is designed to help nodes establish

TABLE II: Notations used.

Notation	Description
D and AS	Newly joined node and its authentication server
$H(\cdot)$	Collision-resistant hash
$SE(\cdot)$	Symmetric encryption
$SD(\cdot)$	Symmetric decryption
c_D	challenge for D by AS
m_D	session-based random for D and AS
y_D	Unique secret of D
c_{AS-D}	challenge for AS by D
T_{Acc}	Accumulated value of AS
ts_1, ts_2	timestamp components

secure multihop communications with the gateway. To establish a secure communication, nodes should be authenticated by a trusted authentication server. In this case, a set of existing IoT nodes inside the network is entrusted with the responsibility of authentication by the gateway in a distributed manner. The gateway also shares a secret key with each of those responsible IoT nodes/authentication servers. The authentication server can be a parent or a non-parent node. In other words, the parent node and the authentication server do not necessarily have to be the same entity; they may be decoupled. Thus, prevents newly joined child nodes, choosing an IoT node as their parent, from overwhelming that parent node with biased authentication load. IoT nodes are assumed to be embedded with a PUF and are capable of performing various cryptographic operations, such as encryption/decryption, XOR, hashing, bitwise AND, and concatenation.

Nodes D and AS are assumed to be the newly joined node and its authentication server, respectively. Additionally, GW represents the gateway symbolically. AS may be located at a singlehop or multihop distance from D . Initially, it is assumed that GW has already selected the authentication server set and has established a shared secret key with each member of the set. Let $AS' = \{AS_1, AS_2, \dots, AS_n\}$ be the set of authentication servers selected by GW , such that $AS \in AS'$. GW has a shared secret key K_{GW-AS} with AS . It is also assumed that D , whose distinct identifier is ID_D , possesses the address of its authentication server AS along with its unique identity ID_{AS} . Further, Table II elaborates symbols and functions incorporated in the following scheme. There are mainly three phases—the node enrolment, the node authentication, and the node session key exchange phase. The following section elaborates on these phases.

A. Node Enrolment Phase

One of the two main phases of the proposed scheme is the node enrolment phase. It is the first phase that is undergone by a newly joined node D with its AS . Before entering this phase, it is assumed that D has the required information regarding AS : the address and unique identity (ID_{AS}) of AS . AS can lie at a singlehop or multihop distance from D . This phase is assumed to be executed in a secure environment by establishing an end-to-end encrypted channel between D and AS . Every newly joined node will execute this phase once until it gets disconnected from the network. The main purpose of this phase is to enrol the unique secret of D (y_D) with

the accumulated value (T_{Acc}) of AS . During the node authentication phase, D can again use y_D to prove its membership with AS , undergoing authentication. This is implemented using the hash-based accumulator mentioned in Section III-B. PUF responses and a session-based random are also communicated between AS and D during this phase, to establish the secure communication of y_D during the authentication. Figure 1 outlines the methodology of this phase. D initiates the phase

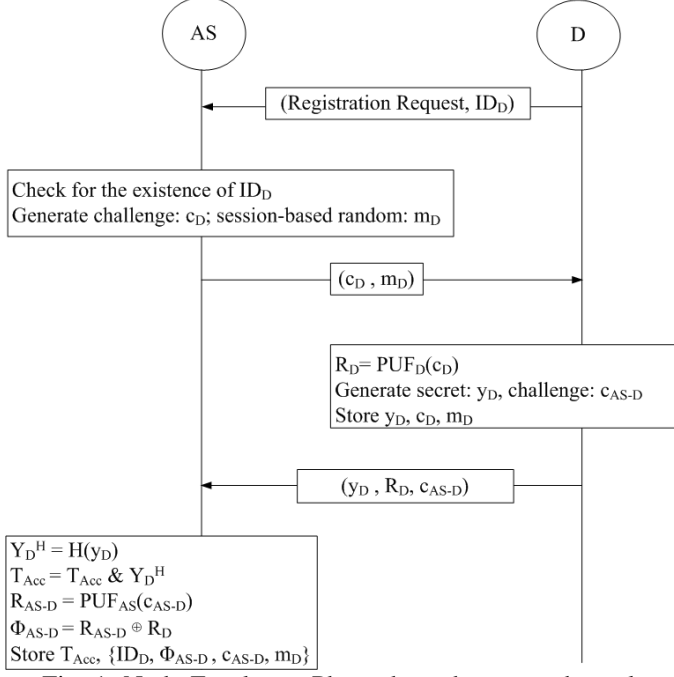


Fig. 1: Node Enrolment Phase through secure channel.

by sending a registration request along with its unique identity ID_D to AS . As a response, AS generates a challenge c_D from the predefined challenges set $\mathbb{C}_D$ and creates a session-based random m_D ; these are then forwarded securely to D . Both of these credentials will act to secure the communication during authentication. With the challenge c_D as an input to its embedded PUF_D , D then obtains a unique response R_D . After this, D generates its own unique secret value y_D , which is also a random number. The security of the entire scheme critically depends on maintaining the confidentiality of y_D . D also picks a challenge c_{AS-D} from the set of challenges $\mathbb{C}_{AS-D}$ and sends it back to AS along with the response R_D and y_D through the pre-established secure channel, maintaining the confidentiality of the secret y_D . AS then inputs the secret y_D to the hash function $H(\cdot)$, generating a tag value Y_D^H . After that, AS performs a bitwise AND operation between the calculated tag and the present accumulated value T_{Acc} . The accumulated value contains the compact, hashed, unique secrets of individual nodes, which are authenticated by AS . In this way, D successfully enrolls itself with AS . Further, AS generates a unique response R_{AS-D} using the received challenge c_{AS-D} in the embedded PUF_{AS} , and calculates Φ_{AS-D} by operating XOR between R_{AS-D} and the received R_D . In this way, R_D is kept in an XORed form for further

secure communications by AS . At the end, AS creates an entry against ID_D and stores the calculated Φ_{AS-D} along with the challenge c_{AS-D} and the received session-based random m_D . On the other hand, D stores its unique secret y_D , the challenge c_D , and the same session-based random m_D .

B. Node Authentication Phase

The main purpose of this phase is to prove the membership of D 's unique secret y_D to AS . Since the authentication phase is usually executed via an open channel, y_D should be communicated in a secure form, preventing it from getting intercepted by adversaries. This security is established through PUF-based challenge-response, producing a secret key R_D , and the session-based random m_D incorporating unpredictability, as shown in Figure 2. Both R_D and m_D are only known by D and AS . The phase is initiated by D , which sends its hashed

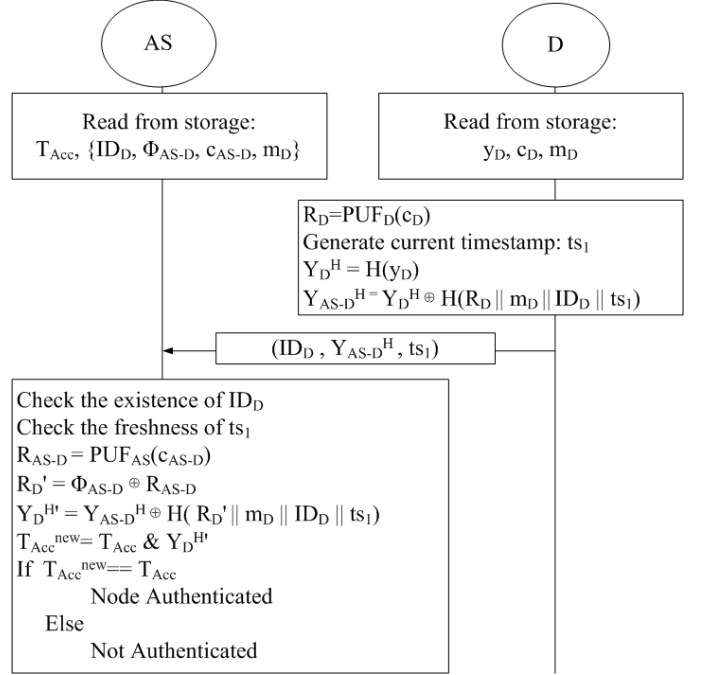


Fig. 2: Node Authentication Phase.

secret Y_D^H in an XORed value (Y_{AS-D}^H). The XORed value is generated using an XOR mask that consists of the hash of R_D regenerated from the challenge c_D and the session-based random m_D . Both c_D and m_D are obtained during enrolment from AS . The current timestamp ts_1 is also attached as an input of the hash along with the unique identity of D (ID_D). The timestamp is used to provide replay protection to the XORed Y_D^H , i.e., Y_{AS-D}^H , which is obtained by operating a lightweight XOR between Y_D^H and the obtained XOR mask ($H(R_D || m_D || ID_D || ts_1)$). The entire message (ID_D , Y_{AS-D}^H , ts_1) is then sent back to AS . An attacker will not be able to obtain Y_D^H without R_D (can only be generated by D) and m_D (only known to D through enrolment).

After receiving the message, AS checks the existence of the entry of D . If it exists, AS un.masks Y_D^H , regenerating the hashed XOR mask using R_D , the session-based random m_D (retrieved from the entry of ID_D), the received ts_1 , and the

unique identity of D (ID_D). R_D is calculated by the XOR operation between the PUF response R_{AS-D} of the challenge c_{AS-D} and Φ_{AS-D} (both c_{AS-D} and Φ_{AS-D} are read from the entry of ID_D). After Y_D^H is reinstated, AS repeats the bitwise AND operation between it and the present accumulated value T_{Acc} . From the properties of the hash-based one-way accumulator (Section III-B), if an element is combined into an accumulated value using bitwise AND and the same operation is again performed between them, it gives back the same accumulated value (mainly due to the property of the bitwise AND operation). In this case, if the new accumulated value, i.e., the accumulated value after the operation, appears to be equal to the previous accumulated value, then it proves that the unique secret of D is already a legitimate member of AS . With the successful operation, the authentication of D is completed by AS .

C. Node Session Key Exchange Phase

This phase is executed by AS to inform GW about the authentication of the newly joined node D at the end of the node authentication phase. AS uses an $auth_token_D$, where it puts the session key (K_{GW-D}) of GW and D . K_{GW-D} is calculated by AS using the hashed output of the input consisting of R_D and m^{new} . R_D is the PUF response of D , which is regenerated by AS during the node authentication and thus, known only by AS and D . m^{new} , on the other hand, is computed by hashing a new random number n_1 generated by AS . The computed m^{new} will also act as the m_D for the next session of authentication between AS and D , as shown in Figure 3, incorporating an update to the authentication secret. The session key, its own identity ID_{AS} , and

resultant input is encrypted using the shared secret key of GW and AS , i.e. K_{GW-AS} (retrieved from storage), generating $auth_token_D$. This $auth_token_D$ then informs GW about the authentication of ID_D along with the session key K_{GW-D} and the completion time of authentication (ts_{auth}). On the other hand, AS also informs m^{new} to D , so that D can compute the session key with GW . The session key is generated with the hash of R_D and the received m^{new} . AS has to send this m^{new} in a secure form, protecting it from adversaries. This is because, if an adversary successfully captures the m^{new} , the probability of it guessing the content of the authentication message from future communications will increase. So, AS masks m^{new} , performing a XOR operation of $H(Y_D^H \parallel m_D \parallel R_D \parallel ID_{AS} \parallel ID_D \parallel ts_2)$ with it to produce m^H . Here, Y_D^H is the hashed form of the unique secret of D , which was known to AS in the node authentication phase. Both the session-based random m_D and the PUF response of R_D (calculated) are known to AS from the authentication phase. The current timestamp ts_2 is used to prevent the message m^H from replay. Obtaining m^H , m^{new} is retrieved by D regenerating the hash and repeating the same XOR operation, but this time with m^H . The input of the regenerated hash is already known by D . In this way, D and AS update their shared session-based random m_D with m^{new} . Alongside, D establishes the session key, K_{GW-D} , with the hash of the concatenated input ($R_D \parallel m_D$), thus preparing itself for secure communication with GW .

V. SECURITY ANALYSIS

This section is dedicated to a detailed security analysis of the proposed lightweight authentication scheme. Both formal and informal analyses have been performed against standard security threats like replay attacks, man-in-the-middle attacks, and offline-guessing attacks.

A. Formal Security Analysis

ProVerif [24] serves as an automated tool that formally verifies the security of cryptographic protocols. The attacker model is based on the Dolev-Yao model [25], where an adversary has full control over the communication channel, meaning it can intercept, modify, delete, and replay any communication through that channel. It analyzes crucial properties including reachability, correspondence, and observational equivalences. To perform the formal evaluation of the proposed scheme elaborated in Section IV, the algorithm is translated into ProVerif queries as presented in Figure 4.

1) Completion of Authentication and Resistance to Replay and Man-in-the-middle attack

This section ensures the successful completion of authentication between D and AS . In ProVerif, this is expressed through event correspondence properties by capturing two levels of authentication:

- *Identity Agreement* (agreement only on the identities of participating entities)
- *Full Agreement* (agreement not only on identities but also on exchanged session-based credentials)

The following events are declared:

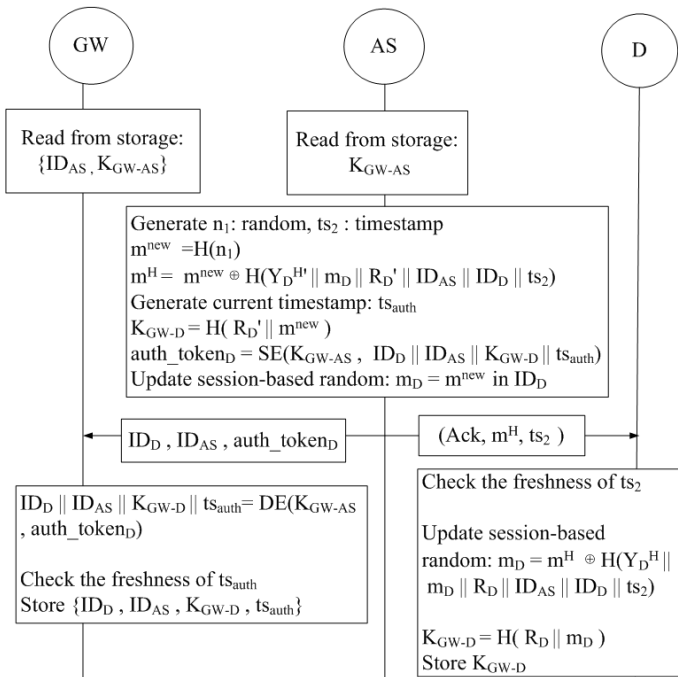


Fig. 3: Node Session Key Exchange Phase.

D 's unique identity (ID_D) are concatenated together, along with the completion timestamp of authentication (ts_{auth}). The

```

sreeparna@sreeparna-HP-ProDesk-600-G6-Microtower-PC: ~/...
Verification summary:

Query inj-event(DeviceEnrolled(id_D_3,id_AS_3)) ==> inj-event(DeviceEnrollmentStarts(id_D_3,id_AS_3)) is true.

Query inj-event(DeviceAuthenticated(id_D_3,id_AS_3)) ==> inj-event(DeviceAuthenticationStarts(id_D_3,id_AS_3)) is true.

Query inj-event(AuthenticationServerEnds(id_AS_3,id_D_3)) ==> inj-event(AuthenticationServerStarts(id_AS_3,id_D_3)) is true.

Query inj-event(AuthenticationEndsFull(id_D_3,id_AS_3,R_D_4,m_D_2)) ==> inj-event(AuthenticationStartsFull(id_D_3,id_AS_3,R_D_4,m_D_2)) is true.

Query inj-event(ServerEnds(id_D_3)) ==> inj-event(DeviceStarts(id_D_3)) is true.

Query not attacker(SecretK_GW_D[]) is true.

Query not attacker(SecretK_GW_D_AS[]) is true.

Weak secret SecretK_GW_D is true.

Weak secret SecretK_GW_D_AS is true.

```

Fig. 4: ProVerif Output.

- 1) event DeviceEnrollmentStarts(ID_D , ID_{AS}) / event DeviceEnrolled(ID_D , ID_{AS})
- 2) event DeviceAuthenticationStarts(ID_D , ID_{AS}) / event DeviceAuthenticated(ID_D , ID_{AS})
- 3) event AuthenticationServerStarts(ID_{AS} , ID_D) / event AuthenticationServerEnds(ID_{AS} , ID_D)
- 4) event AuthenticationStartsFull(ID_D , ID_{AS} , R_D , m_D) / event AuthenticationEndsFull(ID_D , ID_{AS} , R_D , m_D)

Here, the first six events capture agreement only on identities of D and AS , while the last two full events capture agreement on secrets: R_D and m_D , along with identities.

The following injective correspondence assertions must hold to ensure security:

$$\begin{aligned}
&\text{inj-event}(\text{DeviceEnrolled}(ID_D, ID_{AS})) \Rightarrow \\
&\text{inj-event}(\text{DeviceEnrollmentStarts}(ID_D, ID_{AS})) \\
&\text{inj-event}(\text{DeviceAuthenticated}(ID_D, ID_{AS})) \Rightarrow \\
&\text{inj-event}(\text{DeviceAuthenticationStarts}(ID_D, ID_{AS})) \\
&\text{inj-event}(\text{AuthenticationServerEnds}(ID_{AS}, ID_D)) \Rightarrow \\
&\text{inj-event}(\text{AuthenticationServerStarts}(ID_{AS}, ID_D)) \\
&\text{inj-event}(\text{AuthenticationEndsFull}(ID_D, ID_{AS}, R_D, m_D)) \\
&\Rightarrow \text{inj-event}(\text{AuthenticationStartsFull}(ID_D, ID_{AS}, R_D, m_D))
\end{aligned}$$

Satisfaction of these properties confirms that whenever a party believes it has completed the authentication with an entity, it indeed initiated a corresponding session with that same entity. This also proves resistance against replay and man-in-the-middle (MITM) attacks.

2) Reachability and Secrecy of Session Keys

Another critical objective is to verify the secrecy of the session key established between D and GW through AS . ProVerif's reachability analysis tests whether an adversary can obtain the session key. The following queries were specified:

- 1) query attacker(SecretK_GW_D[])
- 2) query attacker(SecretK_GW_D_AS[])

Here, SecretK_GW_D_D and SecretK_GW_D_AS denote test messages encrypted under the session key K_{GW-D} and are

sent over the public channel after authentication. Successful verification ensures that the attacker cannot obtain the session key or recover these messages, thereby proving session key secrecy.

3) Observational Equivalence (OE) Analysis

Finally, we evaluate a class of observational equivalence to confirm the indistinguishability of protocol runs, i.e., Resistance to off-line guessing attacks—checked via weaksecret SecretK_GW_D_D; SecretK_GW_D_AS, confirming that low-entropy secrets cannot be guessed without interaction. Both SecretK_GW_D_D and SecretK_GW_D_AS hold the same meaning as shown in Section V-A2.

B. Informal Analysis

1) Resistance to Replay attack

Replay attacks may occur by the retransmission of an older and/or modified intercepted message. During the node enrolment phase, transmissions are executed through a secure channel, so the adversary does not have access to communications taking place, mitigating the replay attack. In the node authentication and session key exchange phase, the proposed scheme has incorporated timestamp-based freshness check (ts_1 , ts_2 , and ts_{auth}) and session-based randoms (m_D) to prevent replay attacks. As an adversary replays an older/modified intercepted message with outdated/modified timestamp values and session-based randoms, messages will not pass verification, preventing the replay attack.

2) Resistance to Man-in-the-middle attack

A man-in-the-middle (MITM) attack occurs when an adversary intercepts and modifies communications between two legitimate entities without being detected. The node enrolment phase is resistant to the MITM attack, as it is conducted over a secure channel. In the node authentication and session key exchange phase, the use of a one-way collision-resistant hash function and a secure encryption algorithm, along with a unique PUF response (R_D) and session-based random (m_D), prevents adversaries from successfully modifying the intercepted message. Without access to R_D and the updated m_D , any attempt at modification will be caught by participating entities.

VI. PERFORMANCE ANALYSIS

A. Theoretical Performance Analysis

1) Computational Cost

The computational cost is a crucial deterministic factor ensuring the security and efficiency of an authentication scheme. Pycrypto, a Python cryptographic library, is used in a Windows system to measure the computational cost of core cryptographic functions, such as SHA-256 (T_{hash}), AES Encryption/Decryption (T_{aes}), ECC point multiplication/addition (T_{ecc}), and Random generation (T_{rand}).

To execute the node enrolment phase (Figure 1):

- AS computes two random generations (T_{rand}), one PUF operation (T_{puf}), and one hash operation (T_{hash}).
- On the other hand, D executes two random generations (T_{rand}), and one PUF operation (T_{puf}).

Hence, the total computational cost incurred by per node enrolment is: $2T_{puf} + 1T_{hash} + 4T_{rand}$. As shown in Table III,

TABLE III: Comparison of Computational Cost: Enrolment Phase

Scheme	Operations	Cost(ms)
[11]	$1T_{puf} + 2T_{hash} + 6T_{aes}$	0.64
[13]	$5T_{hash} + 2T_{ecc} + 2T_{rand}$	2.17
[14]	$3T_{hash} + 1T_{ecc} + 3T_{rand}$	1.10
[15]	$2T_{puf} + 8T_{hash} + 4T_{aes} + 5T_{rand}$	0.69
[10]	$4T_{rand}$	0.01
Proposed Scheme	$2T_{puf} + 1T_{hash} + 4T_{rand}$	0.15

Benchmark (avg. of 100 iterations):

$T_{rand} = 0.0018$ ms (Random generation), $T_{puf} = 0.0522$ ms (PUF operation), $T_{hash} = 0.0353$ ms (SHA-256), $T_{aes} = 0.0867$ ms (AES), $T_{ecc} = 0.9973$ ms (ECC Mult./Add).

the proposed scheme achieves an enrolment cost of 0.15 ms, outperforming most alternative schemes except [10]. However, as [10] incorporates computationally intensive AES operations during the authentication and key exchange phase (Table IV), its total computational cost (1.22 ms) exceeds that of the proposed scheme (0.71 ms).

Due to the frequent execution of the authentication and key exchange phase, analysis of its computational efficiency becomes inevitable. In the proposed scheme, D and AS (Figure 2, 3) both execute one PUF (T_{puf}) and four hash (T_{hash}) operations individually. Additionally, AS performs one AES encryption (T_{aes}) and one random generation (T_{rand}) along with GW running another AES decryption (T_{aes}). This leads to the total computational cost of the authentication and key exchange phase: $2T_{puf} + 8T_{hash} + 2T_{aes} + 1T_{rand}$. In Table IV, the proposed scheme is said to have a computational cost of 0.56 ms. Thus, its computational efficiency surpasses that of the ECC-based scheme ([11], [13], [14]), and other computationally intensive schemes ([15], [10]). Our scheme performs faster authentication by leveraging a lightweight PUF and hash operations along with a smaller number of AES encryptions.

TABLE IV: Comparison of Computational Cost: Authentication and Key Exchange Phase

Scheme	Operations	Cost(ms)
[11]	$2T_{puf} + 8T_{hash} + 3T_{ecc} + 3T_{rand}$	3.39
[13]	$6T_{hash} + 4T_{ecc}$	4.20
[14]	$2T_{puf} + 12T_{hash} + 2T_{ecc} + 4T_{rand}$	2.53
[15]	$2T_{puf} + 6T_{hash} + 4T_{aes} + 5T_{rand}$	0.67
[10]	$14T_{aes}$	1.21
Proposed Scheme	$2T_{puf} + 8T_{hash} + 2T_{aes} + 1T_{rand}$	0.56

Benchmark (avg. of 100 iterations):

$T_{rand} = 0.0018$ ms (Random generation), $T_{puf} = 0.0522$ ms (PUF operation), $T_{hash} = 0.0353$ ms (SHA-256), $T_{aes} = 0.0867$ ms (AES), $T_{ecc} = 0.9973$ ms (ECC Mult./Add).

2) Communication Cost

Communication cost is a core feature for analyzing the performance of an authentication scheme, based on the number of message exchanges and their sizes (in bits). By assuming that communications are executed with uniform bit lengths, the communication cost of both enrolment and authentication (key exchange phase) is evaluated and compared with other state-of-the-art schemes.

It is assumed that identities/timestamps are 32 bits, along with nonces/hashes/challenges/responses/ciphertexts to be 256 bits, and ECC points as 320 bits [11], ensuring a fair comparison.

During the node enrolment phase, D enrolls itself with AS , executing three message exchanges. First two messages communicate the unique identity of D i.e. ID_D (32 bits), a challenge c_D (256 bits), and session-based random m_D (256 bits). The third communication takes place from D to AS in the form of a unique secret y_D (256 bits), a response R_D (256 bits), and another challenge c_{AS-D} (256 bits). Thus, the total communication cost per enrolment is 1312 bits as presented in Table V. It outperforms schemes like [13] and [15], except [11], [14], [10]. However, the proposed scheme balances this overhead with an efficient authentication and key exchange phase.

TABLE V: Comparison of Communication Cost: Enrolment Phase

Scheme	Number of Messages	Cost (bits)
[11]	2	864
[13]	2	1824
[14]	2	992
[15]	3	1600
[10]	2	1056
Proposed Scheme	2	1312

All schemes assume: temporary IDs and timestamps = 32 bits; random nonces, hash digests, challenges, response, and symmetric ciphertexts = 256 bits; ECC points (P-256) and Public key = 320 bits;

The authentication and key exchange phase is executed frequently using a three-message communication. The first communication takes place from D to AS , where D communicates its unique identity ID_D (32 bits), a hashed value Y_{AS-D}^H (256 bits), and a timestamp ts_1 (32 bits). In the second communication, AS transmits a hashed value m^H (256 bits), and a timestamp ts_2 (32 bits) to D . The third communication takes place between AS and G consisting of identities: ID_{AS} and ID_D (32 bits each), and an $auth_token_D$ (256 bits). Thus, the total communication cost becomes 928 bits, which appears to be best among other schemes in Table VI.

TABLE VI: Comparison of Communication Cost: Authentication and Key Exchange Phase

Scheme	Number of Messages	Cost (bits)
[11]	2	1408
[13]	2	2612
[14]	3	4160
[15]	3	1600
[10]	6	1536
Proposed Scheme	3	928

All schemes assume: temporary IDs and timestamps = 32 bits; random nonces, hash digests, challenge, responses, and symmetric ciphertexts = 256 bits; ECC points (P-256) = 320 bits

B. Simulation-based Performance Analysis

A simulation-based performance evaluation has been performed to detect the efficiency of the proposed authentication framework using Cooja (Contiki-NG) simulator. It has been executed by randomly deploying an IoT network of 100 nodes in the simulator. In this network, 20 nodes are considered to be newly joined nodes. It is also assumed that Node 1 is the

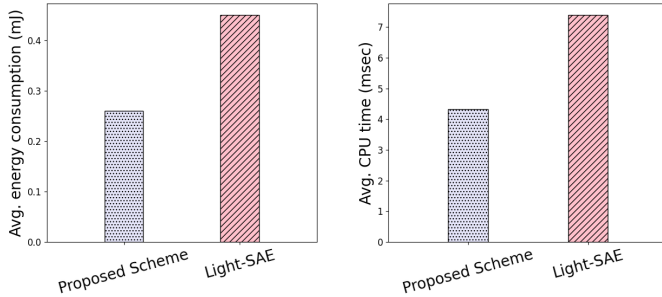
TABLE VII: Simulation Parameters

Parameter	Value
Operating System	Contiki-NG
Mote Type	Cooja Mote
Network Size	100 Nodes
Propagation Model	UDGM (Distance Loss)
Application Layer	CoAP
Network Layer Protocol	RPL Lite
MAC Layer Protocol	CSMA
Simulation Time	75 Seconds
No. of Simulation	10 times

gateway/root node, along with Node 2 and Node 3 as authentication servers. Half of the newly joined nodes are considered to be authenticated by Node 2 and the other half by Node 3 through CoAP. The simulation parameters are elaborated in Table VII. LightSAE [10] is implemented alongside the proposed methodology as the benchmark scheme.

1) Performance Analysis for One-time Authentication.

Initially, the authentication schemes (both the proposed and benchmark Scheme LightSAE [10]) are evaluated by considering the overhead of one-time authentication (enrolment and authentication phase). For 20 newly joined nodes in a 100-node network, the proposed scheme demonstrated a consistent advantage over LightSAE by achieving a reduction of 41.4% and 41.5% with respect to average energy consumption (Figure 5a) and average CPU time (Figure 5b), respectively.



(a) Average Energy consumption (b) Average CPU time
Fig. 5: Performance of one-time authentication: (a) Average energy consumption and (b) Average CPU time.

2) Performance Analysis for Multiple Authentications.

In IoT networks, the authentication phase is executed frequently. So, analyses have been performed by considering multiple executions of the authentication phase. The results highlight the performance efficiency of the proposed scheme in comparison to the baseline scheme, i.e., LightSAE ([10]). Considering the average energy (per authentication) consumption (Figure 6), a reduction of 55%, 33%, 53%, and 40% is demonstrated for 5, 10, 15, and 20 authentications, respectively. Similarly, as shown in Figure 7, the average CPU time (per authentication) is reduced by 54%, 32%, 53%, and 40% for the corresponding authentication rounds. These consistent reductions demonstrate the energy and computational efficiency of the scheme in comparison to the benchmark.

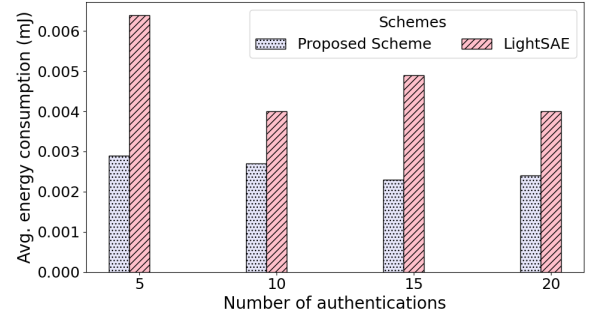


Fig. 6: Average Energy Consumption (per authentication) for Multiple Authentications.

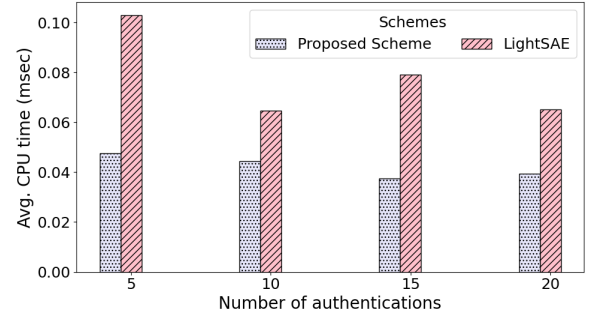


Fig. 7: Average CPU time (per authentication) for Multiple Authentications.

Due to the incorporation of fewer message exchanges and computationally lightweight operations, the proposed scheme has minimized radio activities and computational load compared to the computationally intensive LightSAE.

VII. CONCLUSION

This study elaborated on a lightweight, secure, and decoupled distributed authentication scheme, where IoT nodes (capable of authentication) are authorized by the gateway to perform authentication. The node playing the role of the authentication server may be a parent or a non-parent node, separating the authentication and node joining process. Thus, prevents child nodes from overwhelming their parent with biased authentication load. At the end of authentication, the server mediates in the session key generation between the gateway and the authenticated node, while informing the gateway about the node. The scheme has obtained efficiency and robustness by eliminating secret key storage requirements, incorporating unpredictability in authentication communications, and leveraging lightweight cryptographic operations. Both security and performance analyses are conducted, confirming its suitability for a resource-constrained IoT network.

REFERENCES

- [1] I. Zhou, I. Makhdoom, N. Shariati, M. A. Raza, R. Keshavarz, J. Lipman, M. Abolhasan, and A. Jamalipour, "Internet of Things 2.0: Concepts, Applications, and Future Directions," *IEEE Access*, vol. 9, pp. 70961–71012, 2021.

- [2] B. Kang, D. Kim, and H. Choo, "Internet of Everything: A Large-Scale Autonomic IoT Gateway," *IEEE Transactions on Multi-Scale Computing Systems*, vol. 3, pp. 206–214, 2017.
- [3] N. Lajnef, A. Mami, and F. Derbel, "Applications of Wireless Sensor Networks and IoT Frameworks in Industry 4.0: A Systematic Literature Review," *Sensors*, vol. 22, no. 6, 2022.
- [4] J. Granjal, E. Monteiro, and J. S. Silva, "Security in the Integration of Low-Power Wireless Sensor Networks with the Internet: A Survey," *Ad Hoc Networks*, vol. 24, pp. 264–287, 2015.
- [5] W. Alnahari and M. T. Quasim, "Authentication of IoT Device and IoT Server using Security Key," in *Proceedings of 2021 International Congress of Advanced Technology and Engineering (ICOTEN)*, pp. 1–9, IEEE, 2021.
- [6] M. Khatua, S. Das, and P. M. Rana, "PUF-based Lightweight Mutual Authentication Scheme for IoT-based Smart Grids," in *Proceedings of 2024 IEEE International Conference on Advanced Networks and Telecommunications Systems (ANTS)*, pp. 1–6, IEEE, 2024.
- [7] S. A. Sheik and A. P. Muniyandi, "Secure authentication schemes in cloud computing with glimpse of artificial neural networks: A review," *Cyber Security and Applications*, vol. 1, p. 100002, 2023.
- [8] D. He, Y. Cai, S. Zhu, Z. Zhao, S. Chan, and M. Guizani, "A lightweight authentication and key exchange protocol with anonymity for IoT," *IEEE transactions on wireless communications*, vol. 22, no. 11, pp. 7862–7872, 2023.
- [9] S. Roy, M. H. Mahalat, and B. Sen, "Design and implementation of cost-effective end-to-end authentication protocol for PUF-enabled IoT devices," *IEEE Transactions on Emerging Topics in Computing*, 2025.
- [10] P. Rosa, A. Souto, and J. Cecilio, "Light-SAE: A Lightweight Authentication Protocol for Large-Scale IoT Environments made with Constrained Devices," *IEEE Transactions on Network and Service Management*, vol. 20, no. 3, pp. 2428–2441, 2023.
- [11] O. Alruwaili, M. Tanveer, S. A. Aldossari, S. Alanazi, and A. Armghan, "RAAF-MEC: Reliable and Anonymous Authentication Framework for IoT-enabled Mobile Edge Computing Environment," *Internet of Things*, vol. 29, p. 101459, 2025.
- [12] M. Tiloca, D. Migault, M. Sethi, Z. Zhang, O. Hahm, P. van der Stok, and M. Vucinic, "Security Considerations in the IP-based Internet of Things," Internet-Draft draft-irtf-t2trg-iot-secons-12, Internet Research Task Force (IRTF), 2023. Work in Progress.
- [13] W. Yang, C. Hou, Y. Wang, Z. Zhang, X. Wang, and Y. Cao, "SAKMS: A Secure Authentication and Key Management Scheme for IETF 6TiSCH Industrial Wireless Networks based on Improved Elliptic-Curve Cryptography," *IEEE Transactions on Network Science and Engineering*, vol. 11, no. 3, pp. 3174–3188, 2024.
- [14] S. Li, T. Zhang, B. Yu, and K. He, "A Provably Secure and Practical PUF-based End-to-End Mutual Authentication and Key Exchange Protocol for IoT," *IEEE Sensors Journal*, vol. 21, no. 4, pp. 5487–5501, 2020.
- [15] Y. Zheng, W. Liu, C. Gu, and C.-H. Chang, "PUF-based Mutual Authentication and Key Exchange Protocol for Peer-to-Peer IoT Applications," *IEEE Transactions on Dependable and Secure Computing*, vol. 20, no. 4, pp. 3299–3316, 2022.
- [16] R. Amin *et al.*, "Group authentication protocols for iot: Challenges and solutions," *arXiv preprint arXiv:1909.06371*, Sept. 2019.
- [17] G. Selander, J. P. Mattsson, and F. Palombini, "Ephemeral Diffie-Hellman Over COSE (EDHOC)," RFC 9528, Internet Engineering Task Force, Mar. 2024. Proposed Standard.
- [18] S. Al-Riyami and K. G. Paterson, "Certificateless Public Key Cryptography," in *Proceedings of Advances in Cryptology – ASIACRYPT 2003*, vol. 2894 of *Lecture Notes in Computer Science*, pp. 452–473, Springer, 2003.
- [19] P. Zhang, Y. Wang, G. S. Aujla, A. Jindal, and Y. D. Al-Otaibi, "A Blockchain-Based Authentication Scheme and Secure Architecture for IoT-Enabled Maritime Transportation Systems," *IEEE Transactions on Intelligent Transportation Systems*, vol. 24, no. 2, p. 2322–2331, 2023.
- [20] M. T. Al Ahmed, F. Hashim, S. J. Hashim, and A. Abdullah, "Authentication-Chains: Blockchain-Inspired Lightweight Authentication Protocol for IoT Networks," *Electronics*, vol. 12, no. 4, p. 867, 2023.
- [21] H. Sikarwar, D. Das, and S. Kalra, "Efficient Authentication Scheme Using Blockchain in IoT Devices," in *Proceedings of Advanced Information Networking and Applications (AINA)*, vol. 1151, p. 630–641, Springer, 2020.
- [22] Y. Zheng, Y. Cao, and C.-H. Chang, "A PUF-based Data-Device Hash for Tampered Image Detection and Source Camera Identification," *IEEE Transactions on information forensics and security*, vol. 15, pp. 620–634, 2019.
- [23] H. Weerasena and P. Mishra, "Lightweight Multicast Authentication in NoC-based SoCs," in *Proceedings of 2024 25th International Symposium on Quality Electronic Design (ISQED)*, pp. 1–8, IEEE, 2024.
- [24] B. Blanchet, B. Smyth, V. Cheval, and M. Sylvestre, "ProVerif 2.00: Automatic Cryptographic Protocol Verifier, User Manual and Tutorial," tech. rep., INRIA, 2018. User Manual and Tutorial.
- [25] D. Dolev and A. Yao, "On the security of public key protocols," *IEEE Transactions on Information Theory*, vol. 29, no. 2, pp. 198–208, 2003.